

TP : Projet de mise en production sous contrainte

Owner	 louis reynouard
Tags	
Created time	May 1, 2026 4:41 PM

Vue d'ensemble

Ce projet consiste à déployer un pipeline ML multi-services sous contrainte de ressources. Vous avez plusieurs modèles à faire fonctionner simultanément dans un cluster Kubernetes dont le quota total est fixé et non négociable. L'enjeu n'est pas de construire le meilleur modèle : c'est de produire un système qui fonctionne, tient sous charge, et peut être déployé sur une machine inconnue depuis un simple `git clone`.

Ce projet est évalué en deux temps. Pendant les séances, vous construisez et documentez votre système. À la correction, je clonerai votre repo, lancerai votre commande de déploiement, et exécuterai le script de charge sur ma propre machine. Ce que votre système produit sous cette charge, en direct, est la base principale de votre note.

Chaque décision prise en séance 1 (allocation des ressources, stratégie de déploiement, choix des modèles) a des conséquences directes sur ce que j'observerai pendant le stress test.

Ce qui est imposé à tous

L'architecture minimale comprend trois services containerisés et déployés dans un namespace Kubernetes dédié.

- Un **service de preprocessing** qui reçoit les données brutes, les prépare et les transmet au service d'inférence.

- Un **service d'inférence principal** qui charge un modèle entraîné par vous, répond aux requêtes de prédiction et expose une API REST.
- Un **service de monitoring** qui enregistre les requêtes et les prédictions, et expose des métriques lisibles pendant le stress test (volume, latence, taux d'erreur). Un service de monitoring muet pendant le stress test entraîne la perte des points correspondants.
- Le deuxième modèle peut vivre dans le service d'inférence principal ou dans un quatrième service séparé. C'est une décision d'architecture à justifier dans votre Architecture Decision Record (ADR).
- Le pipeline CI/CD exécute les tests (seuil de couverture 80%), construit les images et les pousse sur Docker Hub. Il échoue si les tests échouent.

Les quotas selon le cas d'usage

Cas d'usage	Quota CPU	Quota mémoire
Cas 1 : Analyse d'images dermatologiques	3500m	5 Gi
Cas 2 : Modération de contenu textuel	3000m	2 Gi
Cas 3 : Prédiction de churn	2500m	1.5 Gi

Les fichiers `quota.yaml` et `limitrange.yaml` ne sont pas fournis : vous devez les écrire vous-mêmes. Voici un exemple pour le Cas 1 :

```
apiVersion: v1
kind: ResourceQuota
metadata:
  name: projet-quota
spec:
  hard:
    requests.cpu: "3500m"
    requests.memory: "5Gi"
    limits.cpu: "3500m"
    limits.memory: "5Gi"
```

Votre Architecture Decision Record (ADR) doit démontrer que la somme de vos requests ne dépasse jamais le quota de votre cas. Ce calcul est obligatoire et vérifié à la correction.

La commande de démarrage Minikube dépend de votre machine. La contrainte est que votre système complet doit tenir dans le quota défini pour votre cas.

Commande recommandée :

```
minikube start --cpus=4 --memory=6144 --driver=docker
```

Choisir son cas d'usage

Cas 1 : Analyse d'images dermatologiques

Contexte métier. Un service de dermatologie reçoit des photographies de lésions cutanées et doit les trier avant l'examen du médecin. Les cas bénins sont déprioritisés, les cas suspects déclenchent une analyse plus fine. Ce projet, mené à bien, amène un bonus de 3 points pour sa complexité.

Dataset. HAM10000 — 10 015 images JPEG de lésions cutanées, 7 classes, annotations CSV. Disponible sur Harvard Dataverse (<https://dataverse.harvard.edu/dataset.xhtml?persistentId=doi:10.7910/DVN/DBW86T>) sans compte, après acceptation des conditions d'utilisation (usage non-commercial). Également sur Kaggle et l'archive ISIC. Licence CC BY-NC 4.0.

Les modèles à entraîner.

Premier modèle : classifieur binaire bénin/malin. MobileNetV2 ou EfficientNet-B0 fine-tuné sur les 7 classes regroupées en deux catégories. Entraînement hors Minikube (Google Colab ou en local), artefact versionné dans le repo.

Deuxième modèle : classifieur multi-classes parmi les 7 types de lésions. Appelé uniquement si le premier modèle détecte une anomalie.

Troisième modèle optionnel (valorisé) : carte de chaleur Grad-CAM sur les cas positifs.

Défi spécifique. Une image JPEG de 800 Ko représente environ 600 Ko à 1 Mo en mémoire décompressée. Sous charge (50 req/min), le preprocessing manipule plusieurs images simultanément. L'allocation mémoire doit tenir compte de cette concurrence.

Cas 2 : Modération de contenu textuel

Contexte métier. Une plateforme doit détecter automatiquement les contenus toxiques avant publication. La latence doit rester sous 500ms.

Dataset. Jigsaw Toxic Comment Classification — 160 000 commentaires Wikipedia annotés sur 6 catégories de toxicité. Téléchargeable directement sur Hugging Face sans compte

(<https://huggingface.co/datasets/thesofakillers/jigsaw-toxic-comment-classification-challenge>), licence CC0. Également disponible sur Kaggle.

Les modèles à entraîner.

Premier modèle : détection binaire toxic/non-toxic. Deux approches possibles à justifier dans l'ADR : TF-IDF + LogisticRegression (~50 Mo en mémoire, latence <50ms) ou DistilBERT fine-tuné (~800 Mo en mémoire, latence 200-400ms). Ce choix est la décision d'architecture centrale de ce cas.

Deuxième modèle : classification fine parmi les 6 catégories pour les textes détectés comme toxiques.

Troisième modèle optionnel (valorisé) : détecteur de langue léger (fasttext langdetect) pour router les textes non-anglophones.

Défi spécifique. Sur un quota de 3 Go, deux instances de DistilBERT consomment ~1.6 Go rien que pour les modèles. Ce calcul doit apparaître dans l'ADR.

Cas 3 : Prédiction de churn et recommandation d'offre

Contexte métier. Un opérateur télécom veut identifier les clients à risque de résiliation et leur proposer une offre en moins de 200ms.

Dataset. Telco Customer Churn (IBM) — 7 043 clients, 21 variables. Très largement disponible : Kaggle, UCI ML Repository, GitHub. CSV de 1 Mo. Licence publique. Pour le modèle de recommandation, générez des données synthétiques à partir du même dataset (5 000 lignes avec 5 catégories d'offres fictives).

Les modèles à entraîner.

Premier modèle : score de churn entre 0 et 1. XGBoost ou Random Forest. Modèle très léger (~20 Mo). Si le score dépasse un seuil configurable, le deuxième modèle est appelé.

Deuxième modèle : recommandation d'offre parmi 5 catégories. Classifieur multi-classes entraîné sur les données synthétiques.

Troisième modèle optionnel (valorisé) : segmentation K-Means déployée comme Kubernetes `CronJob`. Un CronJob est un objet Kubernetes qui exécute un pod selon une planification régulière (par exemple toutes les heures). Contrairement à un Deployment, il consomme des ressources uniquement pendant son exécution — mais ces ressources s'ajoutent temporairement au quota. Le calcul de cette pointe doit apparaître dans l'Architecture Decision Record.

Défi spécifique. Les modèles sont légers mais à 150 req/min, un service Flask mono-threadé risque de saturer. Le défi est de dimensionner le nombre de workers Gunicorn par rapport aux ressources CPU allouées.

Entraîner et valider ses modèles

L'entraînement des modèles se fait **avant la séance 2**, hors Minikube (Google Colab, environnement local, ou tout autre environnement compatible). Minikube est uniquement l'environnement de déploiement des services d'inférence — il n'est pas dimensionné pour l'entraînement.

Les artefacts de modèles (fichiers `.pkl`, `.pt`, `.h5`, etc.) sont versionnés dans le repo sous `models/`. Leur taille totale doit être compatible avec le chargement en mémoire dans le quota alloué.

Chaque modèle doit être accompagné d'une fiche de validation minimale dans `models/README.md` : dataset utilisé, métrique principale obtenue (accuracy, F1, AUC selon le cas), taille de l'artefact, temps d'inférence moyen mesuré en local. Cette fiche est lue pendant la défense — si un modèle n'a aucune métrique documentée, il ne sera pas considéré comme validé.

Déroulé des 5 séances

Séance 1 : Architecture Decision Record

Aucun code attendu. L'ADR est un document de 2 pages maximum déposé sur le repo Git avant la fin de la séance. Il répond sous forme de paragraphes argumentés (pas de listes) aux questions suivantes:

Quel cas d'usage est choisi et pourquoi est-il compatible avec le quota imposé ? Inclure une estimation de la mémoire nécessaire par service, avec le raisonnement explicite (type de modèle, taille des poids, concurrence estimée).

Quel dataset avez-vous identifié, où l'avez-vous trouvé, et quelle est sa licence ?

Comment les services communiquent-ils ? Le deuxième modèle est-il dans le service d'inférence principal ou dans un service séparé ?

Quel outil CI/CD est choisi et pourquoi ? Justification par au moins deux critères concrets.

Quelle stratégie de déploiement (`RollingUpdate` ou `Recreate`) ? Le calcul de la marge disponible dans le quota pour le surge doit apparaître explicitement.

Séance 2 : Containerisation, entraînement et tests

Les modèles doivent être entraînés et leurs artefacts présents dans le repo avant cette séance. N'oubliez pas la fiche de validation `models/README.md` .

Les trois services doivent fonctionner localement avec `docker-compose up` . Les images utilisent des versions pinned pour toutes les dépendances. Les images dépassant 1 Go utilisent un multi-stage build avec justification dans l'ADR.

Les tests couvrent au minimum les fonctions de preprocessing et la logique de routage entre les modèles.

Livrable : images poussées sur Docker Hub avec un tag de version, `docker-compose up --build` fonctionnel depuis un `git clone` sur un environnement vierge. Le `--build` garantit que les images sont reconstruites depuis le code source du repo et ne dépendent pas d'un état local.

Séance 3 : Déploiement Kubernetes et CI/CD

Tous les manifests dans `k8s/` . Un seul `kubectl apply -f k8s/ -n projet-TRIGRAMME` déploie l'intégralité du système.

Les `requests` et `limits` dans les manifests correspondent à des valeurs mesurées avec `kubectl top pods`. Le livrable inclut une capture de `kubectl top pods` justifiant les valeurs choisies.

Livrable : pipeline CI/CD en état `passed` sur `main`, sortie de `kubectl get all -n projet-TRIGRAMME` montrant les pods en état `Running`.

Séance 4 : Challenge de charge en séance

Le script de charge est lancé par chaque étudiant sur son propre système, selon le protocole suivant.

Niveau nominal (10 req/min, 5 min) : relever `kubectl top pods` et les métriques du service de monitoring. Consigner les résultats.

Niveau charge (50 req/min, 5 min) : même procédure, relever `kubectl top pods` toutes les 30 secondes.

Niveau stress (150 req/min, 5 min) : même procédure. Si des pods sont OOMKillés ou throttlés, le noter.

Après les trois tests : identifier la correction la plus impactante et la mettre en oeuvre. Relancer le niveau problématique et mesurer l'effet.

Livrable : tableau comparatif avant/après, description et justification de la correction.

Séance 5 : Stress test sur la machine de l'enseignant + défense

Le stress test se déroule sur ma machine, pendant la correction après l'oral. Je clonerai le repo, exécuterai la commande de déploiement documentée dans le README, et lance le script de charge aux trois niveaux. Ce que le système produit à ce moment-là est la base principale de la note.

Aucune intervention manuelle n'est tolérée après le `git clone`. Un système qui nécessite une manipulation externe pour fonctionner est considéré non reproductible.

L'oral de soutenance dure 15 minutes et se déroule en trois parties.

Partie 1 — Présentation de la solution (5-7 minutes).

Vous présentez votre architecture complète : les services, les modèles, les choix techniques majeurs et leurs justifications. Vous expliquez comment la contrainte de ressources a influencé vos décisions. Support visuel attendu (diagramme d'architecture, tableau de dimensionnement).

Partie 2 — Démo live (5-7 minutes).

Vous montrez le système déployé et fonctionnel : les pods en état Running, une requête d'inférence en direct, les métriques du service de monitoring. Si le système ne démarre pas depuis votre commande, la démo est échouée.

Partie 3 — Questions (3-5 minutes).

Questions sur les décisions prises, les résultats observés pendant le challenge de charge, et la cohérence entre l'Architecture Decision Record et le système produit. Les questions ne sont pas communiquées à l'avance.

Le script de charge

Script fourni et à paramétrer selon le cas d'usage et le niveau.

Placez le script sous `scripts/load_test.py`. Les données doivent être dans `data/` selon les chemins définis.

Structure du repo attendu

```
/
├── .github/
│   └── workflows/
│       └── ci.yml
├── scripts/
│   └── load_test.py
├── data/
│   └── ...
├── models/
└── README.md
```



```
|   └─ ...
|   └─ services/
|       ├── preprocessing/
|           ├── Dockerfile
|           ├── requirements.txt
|           └─ ...
|       ├── inference/
|           ├── Dockerfile
|           ├── requirements.txt
|           └─ ...
|       └─ monitoring/
|           ├── Dockerfile
|           ├── requirements.txt
|           └─ ...
|   └─ k8s/
|       ├── quota.yaml
|       ├── limitrange.yaml
|       ├── preprocessing.yaml
|       ├── inference.yaml
|       └─ monitoring.yaml
|   └─ tests/
|   └─ docker-compose.yml
|   └─ ADR.md
|   └─ README.md
```

Le `README.md` liste dans l'ordre exact les commandes nécessaires pour reproduire le système depuis un terminal vierge. La première section s'intitule "Déploiement".